

063-信号量大题

哲学家就餐问题

- 5 个哲学家围坐在一张圆桌上，两人之间有一把叉子，共 5 把叉子
- 每个哲学家只能拿到自己左右的两把叉子才能吃饭
- 哲学家 思考，饥饿，吃饭

```
Semaphore forks[5]; // 5 把叉子

for (int i = 0; i < 5; i++) {
    forks[i] = 1; // 初始化叉子为可用状态
}

void philosopher(int id) {
    while (true) {
        think();
        P(&forks[id]); // 拿起左叉子
        P(&forks[(id + 1) % 5]); // 拿起右叉子
        eat();
        V(&forks[id]); // 放下左叉子
        V(&forks[(id + 1) % 5]); // 放下右叉子
    }
}
```

- 问题：取两把叉子不是原子操作，可能导致死锁
- 解决
 - 至多允许四个哲学家同时取叉子：增加一个信号量`room=4`，取叉子前先`P(room)`，放下叉子后`V(room)`
 - 每个哲学家取到手边的两把叉子才吃，否则一把叉子也不取：增加一个信号量`mutex=1`，使取两把叉子成为原子操作
 - 奇数号先取左叉子，偶数号先取右叉子：根据`id`判断

管程实现哲学家就餐

```
TYPE philosopher_monitor = monitor
    semaphore forks[5];
    enum {THINKING, HUNGRY, EATING} state[5]; // 哲学家的状态
    for (int i = 0; i < 5; i++) {
        forks[i] = 1; // 初始化叉子为可用状态
        state[i] = THINKING; // 初始化哲学家状态为思考
    }
    condition self[5]; // 哲学家自己的条件变量
    int self_count[5]; // 哲学家自己的条件变量计数器
InterfaceModule IM;

DEFINE pick_forks, put_forks;
```

```

USE wait, signal, enter, leave;

void pickup(int i) {
    enter(IM);                // 进入管程
    state[i] = hungry;        // 表示哲学家想吃饭
    test(i);                  // 尝试进入吃饭状态
    if (state[i] != eating)    // 如果没能成功吃饭
        wait(self[i], self_count[i], IM); // 等待条件变量
    leave(IM);                // 离开管程
}

void putdown(int i) {
    enter(IM);
    state[i] = thinking;      // 放下筷子, 进入思考状态
    test((i - 1) % 5);        // 尝试唤醒左邻
    test((i + 1) % 5);        // 尝试唤醒右邻
    leave(IM);
}

void test(int k) {
    if ((state[(k - 1) % 5] != eating) && // 若左右哲学家都没有在吃饭, 则可以吃饭
        (state[k] == hungry) &&
        (state[(k + 1) % 5] != eating)) {
        state[k] = eating;
        signal(self[k], self_count[k], IM);
    }
}

```

生产者与消费者

- 有 n 个生产者和 m 个消费者
- 缓冲区长度: k
- 只要缓冲区未满, 生产者可以生产
- 只要缓冲区不空, 消费者可以消费
- 同步关系 (具有上下限的资源需要维护两个信号量)
 - 缓冲区没满 -> 生产者生产: 对应信号量: 空闲的缓冲区数
 - 缓冲区不空 -> 消费者消费: 对应信号量: 已放入的产品数
- 互斥关系
 - 生产者和消费者对缓冲区的访问是互斥的: 信号量为 1

$m=n=k=1$

```

Product buffer[1];
Semaphore put = 1; // 可放入的产品数
Semaphore get = 0; // 可取出的产品数

void producer() {
    while (true) {
        Product product = produce();
        P(&put); // 等待可放入的产品数
    }
}

```

```

    buffer[0] = product; // 临界区, 放入产品
    V(&get); // 增加可取出的产品数
}
}

void consumer() {
    while (true) {
        P(&get);
        Product product = buffer[0]; // 临界区, 取出产品
        V(&put);
        consume(product);
    }
}

```

$m=n=1 \quad k>1$

```

Product buffer[k];
Semaphore put = k;
Semaphore get = 0;
int put_ptr = 0, get_ptr = 0;

void producer() {
    while (true) {
        Product product = produce();
        P(&put);
        buffer[put_ptr] = product; // 临界区, 放入产品
        put_ptr = (put_ptr + 1) % k;
        V(&get);
    }
}

void consumer() {
    while (true) {
        P(&get);
        Product product = buffer[get_ptr]; // 临界区, 取出产品
        get_ptr = (get_ptr + 1) % k;
        V(&put);
        consume(product);
    }
}

```

$m, n, k > 1$

```

Product buffer[k];
Semaphore put = k, get = 0;
Semaphore put_mutex = 1, get_mutex = 1; // 对两个`ptr`分别维护互斥状态
int put_ptr = 0, get_ptr = 0;

void producer() {

```

```

while (true) {
    Product product = produce();
    P(&put);
    P(&put_mutex);
    buffer[put_ptr] = product;
    put_ptr = (put_ptr + 1) % k;
    V(&put_mutex);
    V(&get);
}
}

void consumer() {
    while (true) {
        P(&get);
        P(&get_mutex);
        Product product = buffer[get_ptr];
        get_ptr = (get_ptr + 1) % k;
        V(&get_mutex);
        V(&put);
        consume(product);
    }
}

```

管程实现生产者/消费者

```

TYPE ProducerConsumer = monitor
    item Buffer[k];
    int in, out; // 缓冲区的输入输出指针
    int count; // 缓冲区中的产品数
    condition empty, full; // 缓冲区空闲和已放入的产品数
    int empty_count, full_count; // 管程使用的计数器
InterfaceModule IM;

DEFINE append, remove;
USE wait, signal, enter, leave;

void append(item x) {
    enter(IM);
    if (count == k) // 缓冲区已满
        wait(full, full_count, IM);
    Buffer[in] = x; // 放入产品
    in = (in + 1) % k; // 更新输入指针
    count++;
    signal(empty, empty_count, IM); // 增加空闲的缓冲区数
    leave(IM);
}

void remove(item &x) {
    enter(IM);
    if (count == 0) // 缓冲区为空
        wait(empty, empty_count, IM);
}

```

```

    x = Buffer[out]; // 取出产品
    out = (out + 1) % k; // 更新输出指针
    count--;
    signal(full, full_count, IM); // 增加已放入的产品数
    leave(IM);
}

Process Producer() {
    while (true) {
        item x = produce_item(); // 生产产品
        ProducerConsumer.append(x); // 放入缓冲区
    }
}

Process Consumer() {
    while (true) {
        item x;
        ProducerConsumer.remove(x); // 从缓冲区取出产品
        consume_item(x); // 消费产品
    }
}

```

苹果橘子问题 / 农夫猎人问题

- 桌上有一只盘子，每次只能放入一只水果
- 爸爸专向盘子中放苹果，妈妈专向盘子中放桔子
- 一个儿子专等吃盘子中的桔子，一个女儿专等吃盘子里的苹果
- 同步关系：
 - 爸爸放苹果 -> 女儿吃苹果
 - 妈妈放桔子 -> 儿子吃桔子
 - 盘子为空 -> 爸爸和妈妈可以放水果
- 互斥关系：
 - 爸爸、妈妈、儿子、女儿对盘子的访问是互斥的

```

Semaphore apple = 0;
Semaphore orange = 0;
Semaphore plate = 1;
Semaphore mutex = 1;

void father() {
    while (true) {
        P(&plate);
        P(&mutex);
        put_apple();
        V(&mutex);
        V(&apple);
    }
}

void mother() {

```

```

while (true) {
    P(&plate);
    P(&mutex);
    put_orange();
    V(&mutex);
    V(&orange);
}
}

void son() {
    while (true) {
        P(&orange);
        P(&mutex);
        eat_orange();
        V(&mutex);
        V(&plate);
    }
}

void daughter() {
    while (true) {
        P(&apple);
        P(&mutex);
        eat_apple();
        V(&mutex);
        V(&plate);
    }
}

```

- 事实上，可去除mutex，apple orange plate 至多有一个值为1，可以实现互斥
- 如果缓冲区长度大于1，则需要维护互斥状态

管程解决苹果橘子问题

```

TYPE AppleOrange = monitor
    bool isPlateFull;
    enum {APPLE, ORANGE} plateContent; // 盘子中的水果类型
    condition plate, apple, orange;
    int plate_count, apple_count, orange_count; // 管程使用的计数器

InterfaceModule IM;

DEFINE put, eat;
USE wait, signal, enter, leave;

void put(Fruit f) {
    enter(IM);
    if (isPlateFull)
        wait(plate, plate_count, IM);

```

```
isPlateFull = true;
plateContent = f;

if (f == APPLE) {
    signal(apple, apple_count, IM); // 唤醒女儿
} else {
    signal(orange, orange_count, IM); // 唤醒儿子
}

leave(IM);
}

void get(Fruit fruit, Fruit &result) {
    enter(IM);

    if(!full || plate != fruit) {
        if (fruit == APPLE) {
            wait(apple, apple_count, IM); // 等待女儿吃苹果
        } else {
            wait(orange, orange_count, IM); // 等待儿子吃桔子
        }
    }

    result = plateContent; // 获取盘子中的水果
    isPlateFull = false; // 盘子清空

    signal(plate, plate_count, IM); // 唤醒父母可以放水果
    leave(IM);
}

Process Father() {
    while (true) {
        AppleOrange.put(APPLE); // 爸爸放苹果
    }
}

Process Mother() {
    while (true) {
        AppleOrange.put(ORANGE); // 妈妈放桔子
    }
}

Process Son() {
    while (true) {
        Fruit fruit;
        AppleOrange.get(ORANGE, fruit); // 儿子吃桔子
    }
}

Process Daughter() {
    while (true) {
        Fruit fruit;
        AppleOrange.get(APPLE, fruit); // 女儿吃苹果
    }
}
```

```
}  
}
```

读者写者问题

- 允许多个读者可同时对文件执行读操作
- 只允许一个写者往文件中写信息
- 写者在完成写操作之前不允许其他读者或写者工作
- 写者执行写操作前，应让已有的写者和读者全部退出
- 互斥关系：使用 `write_mutex`，保证写者对文件的独占访问
 - 写-写
 - 写-读

读者优先

- 必须等所有读者完成后，才能让写者工作

```
Semaphore write_mutex = 1;  
  
int read_count = 0;  
Semaphore read_count_mutex = 1;  
  
void write() {  
    while (true) {  
        P(&write_mutex);  
        write_to_file();  
        V(&write_mutex);  
    }  
}  
  
void read() {  
    while (true) {  
        P(&read_count_mutex);  
        if (read_count == 0)  
            P(&write_mutex);  
        read_count++;  
        V(&read_count_mutex);  
  
        read_from_file();  
  
        P(&read_count_mutex);  
        read_count--;  
        if (read_count == 0)  
            V(&write_mutex);  
        V(&read_count_mutex);  
    }  
}
```


读写公平

- 增加`queue`，使请求排队完成
- 队列不维护具体的资源，只维护请求的顺序
- 每个`read()`和`write()`在一开始入队，完成可能影响其他进程的逻辑后即可出队（对于`read()`来说是更改完`read_count`，和`read_from_file()`无关）

```
Semaphore write_mutex = 1;
Semaphore read_count_mutex = 1;
Semaphore queue = 1;

int read_count = 0;

void write() {
    while (true) {
        P(&queue);
        P(&write_mutex);

        write_to_file();

        V(&write_mutex);
        V(&queue);
    }
}

void read() {
    while (true) {
        P(&queue);
        P(&read_count_mutex);
        if (read_count == 0)
            P(&write_mutex);
        read_count++;
        V(&read_count_mutex);
        V(&queue);

        read_from_file();

        P(&read_count_mutex);
        read_count--;
        if (read_count == 0)
            V(&write_mutex);
        V(&read_count_mutex);
    }
}
```

写者优先

```
Semaphore read_mutex = {1, NULL};
Semaphore read_try = {1, NULL};
Semaphore write_mutex = {1, NULL};
```

```

Semaphore read_count_mutex = {1, NULL};
Semaphore write_count_mutex = {1, NULL};

int read_count = 0;
int write_count = 0;

void write() {
    P(&write_count_mutex);
    write_count++;
    if (write_count == 1)
        P(&read_mutex);
    V(&write_count_mutex);

    P(&write_mutex);
    write_to_file();
    V(&write_mutex);

    P(&write_count_mutex);
    write_count--;
    if (write_count == 0)
        V(&read_mutex);
    V(&write_count_mutex);
}

void read() {
    P(&read_try);
    P(&read_count_mutex);
    if (read_count == 0)
        P(&write_mutex);

    read_count++;
    V(&read_count_mutex);
    V(&read_try);

    read_from_file();

    P(&read_count_mutex);
    read_count--;
    if (read_count == 0)
        V(&write_mutex);

    V(&read_count_mutex);
}

```

管程实现读者问题

```

TYPE read-write=monitor
    int rc, wc; // 程序员使用的读写计数器
    rc = 0; wc = 0;
    condition R, W;
    R = 0; W = 0;

```

```
int R_count, W_count; // 和信号量对应的，管程使用的计数器
R_count = 0; W_count = 0;
InterfaceModule IM;

DEFINE start_read, end_read, start_write, end_write;
USE wait, signal, enter, leave;

void start_read()
{
    enter(IM);
    if (wc > 0)
        wait(R, R_count, IM);
    rc++;
    signal(R, R_count, IM);
    leave(IM);
}

void end_read()
{
    enter(IM);
    rc--;
    if (rc == 0)
        signal(W, W_count, IM);
    leave(IM);
}

void start_write()
{
    enter(IM);
    wc++;
    if (rc > 0 || wc > 1)
        wait(W, W_count, IM);
    leave(IM);
}

void end_write()
{
    enter(IM);
    wc--;
    if (wc == 0)
        signal(R, R_count, IM);
    else
        signal(W, W_count, IM);
    leave(IM);
}

process Reader()
{
    start_read();
    // Read data
    end_read();
}

process Writer()
```

```
{
    start_write();
    // Write data
    end_write();
}
```

睡眠的理发师问题

- 理发店里有1位理发师、1把理发椅和n把供等候理发的顾客坐的椅子
- 如果没有顾客，理发师便在理发椅上睡觉
- 一个顾客到来时，它必须叫醒理发师
- 如果理发师正在理发时又有顾客来到，则如果有空椅子可坐，就坐下来等待，否则就离开
- 同步关系
 - 顾客叫醒理发师 -> 理发师开始理发
- 互斥关系
 - 一个时间段只能有一个顾客在理发椅上

```
Semaphore awake = 0; // 理发师被叫醒
Semaphore barber_chair_mutex = 1; // 理发椅互斥
int waiting_customers = 0; // 等候区顾客数
int WAITING_AREA_SIZE = n; // 等候区大小
Semaphore waiting_area_mutex = 1; // 等候区互斥

void barber() {
    while (true) {
        P(&awake);

        P(&waiting_area_mutex);
        waiting_customers--;
        V(&waiting_area_mutex);

        V(&barber_chair_mutex);
        cut_hair();
    }
}

void customer() {
    P(&waiting_area_mutex);
    if (waiting_customers < WAITING_AREA_SIZE) {
        waiting_customers++;
        V(&waiting_area_mutex);

        V(&awake);
        P(&barber_chair_mutex);
        get_haircut();
    } else {
        V(&waiting_area_mutex);
    }
}
```

银行业务问题

- n 个柜台，等候区无限
- 每个用户进入银行后取号并等候
- 业务员空闲时叫号
- 同步关系
 - 顾客必须等待有储蓄员空闲才能办理业务
 - 储蓄员必须等待有顾客到达才能服务
 - （不能使用互斥，会触发忙等）

```
int current_customer = 0; // 当前号码
int waiting_customers = 0; // 总顾客数
Semaphore staff_done = 0, customer_ready = 0; // 柜员和顾客的同步
Semaphore customer_count_mutex = 1; // 对 current_customer 和 waiting_customers
使用同一把锁

void staff() {
    while (true) {
        P(&customer_ready);

        // 运行到此处一定存在需要被服务的用户
        P(&waiting_mutex);
        waiting_customers--;
        int customer_id = current_customer++; // 叫号
        V(&waiting_mutex);

        saving(customer_id);

        V(&staff_done);
    }
}

void customer() {
    P(&waiting_mutex);
    waiting_customers++;
    V(&waiting_mutex);

    V(&customer_ready);
    P(&staff_done);
}
```

缓冲区管理

- 将字符读入 Buffer, `Buffer.length = 80`
- n 个进程读入字符到 Buffer
- 1 个进程在 Buffer 满后，从 Buffer 中取走字符
- 同步关系
 - 读满 80 个字符 -> 输出进程取走
 - 缓冲区未满 -> 读入进程可以继续读入字符

- 互斥关系
 - 每次只能有一个进程对 Buffer 进行读/写

```
char buffer[80];
int char_count = 0;
Semaphore buffer_mutex = 1;
Semaphore buffer_empty = 80;
Semaphore buffer_full = 0;

void input_process() {
    char input_char = get_input_char();
    P(&buffer_empty);

    P(&buffer_mutex);
    buffer[char_count++] = input_char;
    if (char_count == 80) {
        V(&buffer_full);
    }
    V(&buffer_mutex);
}

void output_process() {
    while (true) {
        P(&buffer_full);
        P(&buffer_mutex);
        char output[80];
        memcpy(output, buffer, 80);
        char_count = 0;

        V(&buffer_mutex);

        handle_output(output);

        for (int i = 0; i < 80; i++) {
            V(&buffer_empty);
        }
    }
}
```

售票问题

- 售票员关好门应通知司机开车，然后售票员进行售票
- 当汽车已经停下，售票员才能开门上下客，故司机停车后应该通知售票员
- 司机：1名，售票员：2名
- 同步关系
 - **全部**售票员关门后 -> 司机开车
 - 司机停车 -> 售票员开门

```

Semaphore drive[2] = {1, 1};
Semaphore stop[2] = {0, 0};

void driver() {
    while (true) {
        P(&drive[0]);
        P(&drive[1]);

        drive_car();
        park_car();

        V(&stop[0]);
        V(&stop[1]);
    }
}

void conductor(int id) {
    while (true) {
        close_door();
        V(&drive[id]);

        sell_ticket();

        P(&stop[id]);
        open_door();
    }
}

```

吸烟者问题

- 有三种吸烟材料：烟草、纸张、火柴
- 有三名吸烟者，每人有一种材料
- 有一名配料员，负责将两种材料放在桌上
- 每名吸烟者只能在桌上有自己缺少的两种材料时才能吸烟
- 同步关系
 - 配料员放材料 -> 吸烟者开始吸烟
 - 吸烟者吸烟完成 -> 配料员可以放材料

```

Semaphore smoker[3] = {0, 0, 0};
Semaphore finish_smoking = 1;

void supplier() {
    while (true) {
        int material1 = rand() % 3;
        int material2 = rand() % 3;
        if (material1 == material2) {
            continue; // 确保两种材料不同
        }
    }
}

```

```

    P(&finish_smoking);
    put_material(material1, material2);

    int missing = 3 - material1 - material2; // 0+1+2=3
    V(&smoker[missing]);
}
}

void smoker(int id) {
    while (true) {
        P(&smoker[id]);
        smoke(id);
        V(&finish_smoking);
    }
}

```

独木桥问题

基础版

- 独木桥东西双向均来车
- 若桥上无车，允许一方通行
- 当一方车辆全部通过后，另一方车辆才能通行
- 同步关系：一方车辆通过 -> 另一方车辆可以通行
- 互斥关系：一次只能有一方车辆在桥上

```

int to_west = 0, to_east = 0;
Semaphore bridge = 1, west_wait = 0, east_wait = 0;
Semaphore to_west_mutex = 1, to_east_mutex = 1;

void westbound() {
    P(&to_west_mutex);
    to_west++;
    if (to_west == 1)
        P(&west_wait);
    V(&to_west_mutex);

    P(&bridge);
    pass_bridge();
    V(&bridge);

    P(&to_west_mutex);
    to_west--;
    if (to_west == 0)
        V(&east_wait);
    V(&to_west_mutex);
}

void eastbound() {
    P(&to_east_mutex);
    to_east++;

```



```

    if (to_east == 1)
        P(&east_wait);
    V(&to_east_mutex);

    P(&bridge);
    pass_bridge();
    V(&bridge);

    P(&to_east_mutex);
    to_east--;
    if (to_east == 0)
        V(&west_wait);
    V(&to_east_mutex);
}
// 事实上不用 `bridge`, 只需要 `west_wait` 和 `east_wait` 即可

```

桥上车辆上限

- 增加限制：桥面上最多k辆车，将bridge改为信号量k即可

分组通过

- 独木桥东西双向均来车
- 若桥上无车，允许一方通行
- 双方车辆3辆一组，以组为单位通行
- 同步关系
 - 一方车辆达到3辆 -> 该方车辆通行
 - 一方车辆通过 -> 另一方车辆可以通行
- 互斥关系：一次只能有一方车辆在桥上

```

int to_west = 0, to_east = 0;
int west_passed = 0, east_passed = 0; // 增加统计组中车辆数量的变量，和 `to_west`
`to_east` 共用锁

Semaphore bridge_west = 3, bridge_east = 3;
Semaphore west_wait = 0, east_wait = 0;
Semaphore to_west_mutex = 1, to_east_mutex = 1;

void westbound() {
    P(&bridge_west);
    P(&to_west_mutex);
    to_west++;
    if (to_west == 1)
        P(&west_wait);
    V(&to_west_mutex);

    pass_bridge();
    V(&bridge_east);

    P(&to_west_mutex);
}

```

```

    to_west--;
    west_passed++;
    if (to_west == 0 && west_passed == 3) {
        west_passed = 0; // 重置计数
        V(&east_wait);
    }
    V(&to_west_mutex);
}

void eastbound() {
    P(&bridge_east);
    P(&to_east_mutex);
    to_east++;
    if (to_east == 1)
        P(&east_wait);
    V(&to_east_mutex);

    pass_bridge();
    V(&bridge_west);

    P(&to_east_mutex);
    to_east--;
    east_passed++;
    if (to_east == 0 && east_passed == 3) {
        east_passed = 0; // 重置计数
        V(&west_wait);
    }
    V(&to_east_mutex);
}
// 事实上不用 `bridge`, 只需要 `west_wait` 和 `east_wait` 即可

```

阻止过桥

- 当一方提出过桥时，应阻止对方未上桥的后继车辆
- 新增互斥关系：一方车辆提出过桥 -> 阻止对方未上桥的后继车辆

```

int to_west = 0, to_east = 0;
Semaphore bridge = 1, stop = 1, west_wait = 0, east_wait = 0;
Semaphore to_west_mutex = 1, to_east_mutex = 1;

void westbound() {
    P(&stop); // 阻止对方未上桥的后继车辆
    P(&to_west_mutex);
    to_west++;
    if (to_west == 1)
        P(&west_wait);
    V(&to_west_mutex);
    V(&stop); // 允许对方车辆上桥

    P(&bridge);
    pass_bridge();
}

```

```
V(&bridge);

P(&to_west_mutex);
to_west--;
if (to_west == 0)
    V(&east_wait);
V(&to_west_mutex);
}

void eastbound() {
    P(&stop); // 阻止对方未上桥的后继车辆
    P(&to_east_mutex);
    to_east++;
    if (to_east == 1)
        P(&east_wait);
    V(&to_east_mutex);
    V(&stop); // 允许对方车辆上桥

    P(&bridge);
    pass_bridge();
    V(&bridge);

    P(&to_east_mutex);
    to_east--;
    if (to_east == 0)
        V(&west_wait);
    V(&to_east_mutex);
}
// 事实上不用 `bridge`, 只需要 `west_wait` 和 `east_wait` 即可
```